

Architectural Specification and System Handoff: On-Device Machine Learning and Cognitive Application Platform

Architecture Overview & Subsystem Integration

Building modern, high-performance desktop applications requires combining local machine learning inference, real-time audio processing, offline linguistic analysis, and personalized cognitive scheduling into a cohesive software architecture. Modern user expectations demand near-zero latency, complete data privacy, and full offline operational capabilities. The architecture detailed in this specification satisfies these demands by leveraging a Rust backend runtime paired with a Tauri v2 shell, orchestrating specialized native libraries and hardware-accelerated inference pipelines.

The platform is structured into five core functional subsystems, coordinated through a multi-threaded asynchronous architecture. At the perimeter, the user interface shell provides a responsive desktop environment powered by Tauri v2. The platform handles audio acquisition and text-to-speech rendering via ONNX Runtime bindings using the ort crate, driving local Wav2Vec2 automatic speech recognition (ASR) and Piper neural text-to-speech (TTS) models. Cognitive scheduling and spaced repetition memory modeling are managed by the native Rust implementation of the Free Spaced Repetition Scheduler algorithm (fsrs-rs). In-memory grammar analysis and localized text linting are handled by harper-core, while structured application state and review logs are persistently stored using an embedded SQLite database engine with automated migration capabilities.

Subsystem Domain	Primary Library / Technology	Core Responsibility	Key Features & Performance Targets
Application Shell & Host	Tauri v2	Process isolation, UI rendering, native OS capabilities	Security capabilities, window management, dynamic URI protocol
IPC & Data Transport	tauri::ipc::Channel	High-throughput binary audio streaming between UI and Rust	Zero-copy byte buffer transfer, eliminates JSON-RPC overhead
Neural Speech Inference	ort (v2.0-rc series), orp	Wav2Vec2 ASR transcription and Piper TTS voice synthesis	Cross-platform Hardware Acceleration (CUDA, CoreML, DirectML)
Cognitive Memory State	fsrs-rs (FSRS-6)	Memory state prediction (S, D, R), review interval scheduling	21-parameter optimization, Cost ADR cost-conditioned policies
Linguistic Engine	harper-core (v0.62)	Zero-latency local grammar linting and transcript analysis	Offline execution, zero network overhead, deterministic memory usage

Subsystem Domain	Primary Library / Technology	Core Responsibility	Key Features & Performance Targets
Persistent Storage	tauri-plu [span_33](start_span)[span_33](end_span)[span_40](start_span)[span_40](end_span)]gin-sql / SQLite	Relational storage for user history, cards, and parameters	Write-Ahead Logging (WAL) mode, declarative schema migrations

The unified Rust backend guarantees memory safety, fine-grained thread scheduling, and cross-platform hardware optimization. By isolating compute-heavy workloads into dedicated thread pools, the main GUI webview remains responsive at 60 FPS, maintaining low CPU and memory footprints across diverse hardware targets.

Subsystem 1: User Interface Shell & High-Throughput IPC (Tauri v2)

The application shell utilizes Tauri v2 to bridge the Webview frontend runtime with the native Rust operating system backend. Tauri v2 employs a multi-process security architecture where the frontend webview runs in a sandboxed context, constrained by explicit fine-grained permissions defined in application capability manifests. Access to platform capabilities—such as storage, dialogs, and native hardware interfaces—is denied by default and explicitly granted through command scopes.

Inter-Process Communication (IPC) within the application relies on three distinct message-passing patterns depending on payload size, directionality, and latency constraints. Standard, request-response control operations (e.g., updating user settings, querying metadata) utilize typed Tauri commands backed by JSON serialization. Asynchronous status broadcasts and system state updates from the backend to the frontend utilize the webview event emitter system.

Data flows across three primary channels within the Tauri host: discrete JSON commands originate from the webview layer to trigger synchronous or asynchronous execution in the Rust host process; background system events are pushed back to webview event listeners; and continuous, binary stream buffers pass directly through dedicated `tauri::ipc::Channel` instances. When transmitting high-frequency audio data—such as real-time 16 kHz 32-bit floating-point PCM audio frames captured from the client microphone or generated by the speech synthesis engine—standard JSON serialization introduces unacceptable CPU utilization and latency due to base64 encoding and string escaping. To bypass this bottleneck, the platform mandates the use of `tauri::ipc::Channel` for streaming raw binary data buffers directly across the IPC boundary.

IPC Mechanism	Primary Use Case	Payload Format	Latency Characteristics	Throughput Capacity
Tauri Command (invoke)	UI triggers discrete Rust function execution	JSON-RPC Serialized Data	Low-Medium (~1-5 ms)	Moderate (<1 MB per request)
Tauri Event (emit)	Backend pushes asynchronous status updates	JSON Payload	Low-Medium (~2-8 ms)	Moderate (Event namespace scope)
Tauri IPC Channel	Continuous bidirectional	Typed Binary Buffers / Raw	Ultra-Low (<0.5 ms)	High (>50 MB/s continuous)

IPC Mechanism	Primary Use Case	Payload Format	Latency Characteristics	Throughput Capacity
(Channel)	streaming (Audio PCM)	Bytes		stream)

The binary channel mechanism creates a direct point-to-point data pipe between the Webview and the Rust host process. Audio samples captured in the UI via the Web Audio API are written directly into an ArrayBuffer and sent across the channel, where Rust receives contiguous byte slices ready for immediate ingestion into neural inference models.

Subsystem 2: On-Device Speech Recognition & Synthesis Engines (ort & Piper)

On-device machine learning inference is powered by Microsoft's ONNX Runtime via the ort Rust library (v2.0-rc series). The ort crate provides thread-safe, idiomatic Rust bindings to the underlying C API, enabling direct hardware acceleration across diverse host platform targets. To eliminate binary bloat in production distribution packages, applications integrating ort inside downstream libraries must configure Cargo feature flags carefully. Disabling default features prevents bringing unwanted model-fetching code into release builds. Hardware acceleration is managed dynamically by enabling execution providers (EPs) that align with target client graphics architectures.

Execution Provider (EP)	Target Hardware Architecture	Runtime Dependencies	Primary Fallback Strategy
cuda / tensorrt	NVIDIA Maxwell (7xx series) and newer GPUs	CUDA v11.6+, TensorRT v8.4+	DirectML or CPU (oneDNN/OpenMP)
coreml	Apple Silicon (M-series) and iOS devices	Native macOS/iOS SDK Frameworks	CPU Execution Provider
directml	Windows x86/x64 with DirectX 12 compatible GPUs	DirectX 12 Runtime Environment	CPU Execution Provider
openvino / onednn	Intel 6th+ Gen Core CPUs and integrated GPUs	OpenVINO Runtime / oneDNN Libraries	Standard CPU Execution Provider
xnnpack	Mobile ARM and WebAssembly environments	Built-in XNNPACK backends	Standard CPU Execution Provider

The automatic speech recognition (ASR) engine loads Wav2Vec2 models converted to the Open Neural Network Exchange format (e.g., facebook/wav2vec2-base-960h or language-specific VoxPopuli models). Audio streams received via the Tauri IPC channel are framed into 16,000 Hz single-channel 32-bit float vectors. Pre-processing layers normalize wave amplitudes and construct dynamic tensor shapes. The ort::Session runs inference over the input audio frames, outputting probability distributions over the model vocabulary defined in voc[span_119](start_span)[span_119](end_span)[span_121](start_span)[span_121](end_span)]a b.json. Greedy decoding or language-model-guided beam search then maps token IDs to written transcriptions.

Text-to-Speech (TTS) synthesis is powered by piper-rs (or piper-tts-rs), which encapsulates

Piper TTS models trained on Hugging Face model repositories (rhasspy/piper-voices). The speech synthesizer consumes text input, normalizes punctuation, converts characters to phoneme sequences using embedded phonemization libraries (e.g., `espeak-rs`), and executes the underlying ONNX voice model. The model returns raw 22,050 Hz or 16,000 Hz 1-channel PCM float audio chunks. These chunks are streamed back to the UI through a Tauri IPC channel or served via a custom asynchronous URI protocol stream (`tauri::Builder::register_asynchronous_uri_scheme_protocol`) to allow native audio element playback without disk serialization overhead.

To manage operational complexity and streamline pipeline design, pre- and post-processing steps—such as tokenization, feature extraction, tensor formatting, and post-inference decoding—can be wrapped using modular pipeline frameworks like `orp`. Built upon `ort` and the composable crate, `orp` allows developers to chain distinct ONNX models and transformation steps into unified, reproducible execution graphs.

Subsystem 3: Adaptive Spaced Repetition Engine (fsrs-rs)

Cognitive review scheduling is governed by the native Rust implementation of the Free Spaced Repetition Scheduler algorithm (fsrs-rs), based on the DSR (Difficulty, Stability, Retrievability) memory model. Unlike legacy SM-2 algorithms that rely on heuristic ease factors, FSRS uses a 21-parameter differential model to accurately predict memory stability decay curves.

Memory state within fsrs-rs is defined by three fundamental variables:

1. **Difficulty (D)**: Material complexity on a scale from 1.0 to 10.0.
2. **Stability (S)**: Time duration (in days) required for the recall probability to decay to 90% (0.9).
3. **Retrievability (R)**: Probability of successfully recalling the item after t days since the previous review.

The mathematical formulation for Retrievability $R(t)$ as a function of elapsed time t and Stability S is expressed as:

$$R(t) = \left(1 + \text{factor} \cdot \frac{t}{S}\right)^{-\text{decay}}$$

Where decay represents the 21st parameter of the FSRS-6 model (defaulting to 0.1542), and factor is a mathematical constant derived from the targeted 90% stability baseline:

$$\text{factor} = 0.9^{-\frac{1}{\text{decay}}} - 1$$

Under standard FSRS-6 defaults, $\text{factor} \approx 0.9804$, ensuring that when elapsed time $t = S$, retrievability evaluates exactly to $R(S) = 0.90$.

Historical review logs enter the optimization framework through the `compute_parameters` function, which leverages Rayon-based multi-threading to parallelize cost functions and fit the 21 parameters to individual user data. Once optimized, these parameters update the operational FSRS scheduling engine. During routine flashcard evaluation, the engine consumes the card's existing state—comprising Difficulty, Stability, and elapsed time—and calculates discrete next-state intervals for each potential user response: Again (rating 1), Hard (rating 2), Good (rating 3), and Easy (rating 4).

Field / Parameter	Data Type	Description	Operational Scope
Stability (S)	f32 / Days	Days until recall probability drops to 90%	Core memory state tracking

Field / Parameter	Data Type	Description	Operational Scope
Difficulty (D)	f32 / Range [1-10]	Dynamic card complexity rating	Core memory state tracking
Desired Retention	f32 / Range [0.7-0.98]	Targeted recall probability baseline (default: 0.90)	Scheduler interval calculation
Review History Logs	Vec<FSRSItem>	Array of past reviews containing grade and Δt	Input for offline optimization
Cost ADR Policy	CostAdrPolicy	15-parameter cost-conditioned retention optimization	Single-user workload simulation

The system executes scheduling operations using `FSRS::default()` or custom-fitted parameter sets. When a review event occurs, the host application passes the current card's `MemoryState` and elapsed days since last review into `fsrs.next_states()`, receiving post-review memory states and exact day intervals for each rating button (1: Again, 2: Hard, 3: Good, 4: Easy). For high-performing personalization, `fsrs-rs` includes an offline parameter optimization engine. By passing historical review logs (`FSRSItem`) into `compute_parameters`, the engine optimizes all 21 parameters using CPU multi-threading via `Rayon`. Furthermore, the system includes support for the Cost ADR policy (`experimental_cost_adr`), which calculates a cost-conditioned desired retention policy, balancing daily study time budgets against long-term memory retention goals.

Subsystem 4: Local Linguistic Analysis & Grammar Engine (harper-core)

To provide real-time grammatical checks, spellchecking, and style critique without relying on external cloud APIs, the backend incorporates `harper-core` (v0.62 series). Developed specifically as an offline, privacy-focused grammar checking engine for developers, `harper-core` processes unformatted raw text, markdown structures, and ASR transcript outputs locally. Text evaluation follows a strictly deterministic, isolated path within the Rust runtime. Raw audio transcript strings produced by the speech recognition engine, or textual inputs provided by the user, pass directly into the `harper-core` language engine operating in host memory. The engine routes the payload through syntax parsers, heuristic rule sets, and embedded lexicons without invoking external processes or network calls. The output yields a diagnostic stream containing character byte spans, issue severity classifications, structural error messages, and replacement suggestions, which are then delivered to the user interface or auto-correction pipelines. Integrating `harper-core` directly inside the native Rust runtime provides several advantages over external HTTP services or webview-bound JavaScript checkers:

- **Zero Network Latency:** Executed entirely in-memory within native Rust worker threads, completing document parses in sub-millisecond durations.
- **Strict Privacy Isolation:** User writing and audio transcripts never leave local device memory, maintaining strict compliance with local-first security boundaries.
- **Deterministic Resource Bound:** Unlike heavy neural language models, `harper-core` relies on compiled rule sets and optimized dictionary data structures, preserving GPU VRAM and CPU cycles for simultaneous ML inference workloads.

When the speech recognition engine returns transcribed text, the string is routed to harper-core to check for syntax errors, common speech-to-text misrecognitions, and grammatical inconsistencies. Returned diagnostics—containing exact character byte spans, issue descriptions, and suggested corrections—are pushed to the user interface via Tauri events.

Subsystem 5: Embedded Data Persistence & Migration Framework

Persistent application state, scheduling parameters, user cards, and review logs are managed locally via SQLite. SQLite is integrated using tauri-plugin-sql or direct embedded crates such as rusqlite / sqlx.

To maintain dynamic multi-threaded read and write performance alongside background neural processing, the database connection pool configures Write-Ahead Logging (WAL) mode automatically. In WAL mode, read operations execute concurrently without blocking ongoing review log writes, maintaining high performance during batch review operations.

Database migrations are declared programmatically during plugin initialization using `add_migrations`. This pattern ensures schema evolution across client versions occurs seamlessly upon application startup.

```
-- Migration 1: Core Card Management and Scheduling Schema
```

```
CREATE TABLE IF NOT EXISTS cards (  
    id TEXT PRIMARY KEY NOT NULL,  
    deck_id TEXT NOT NULL,  
    content_front TEXT NOT NULL,  
    content_back TEXT NOT NULL,  
    created_at INTEGER NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS card_memory_states (  
    card_id TEXT PRIMARY KEY NOT NULL,  
    stability REAL NOT NULL,  
    difficulty REAL NOT NULL,  
    last_review_date INTEGER NOT NULL,  
    FOREIGN KEY(card_id) REFERENCES cards(id) ON DELETE CASCADE  
);
```

```
-- Migration 2: FSRS Review History Tracking Schema
```

```
CREATE TABLE IF NOT EXISTS review_logs (  
    id TEXT PRIMARY KEY NOT NULL,  
    card_id TEXT NOT NULL,  
    rating INTEGER NOT NULL, -- 1: Again, 2: Hard, 3: Good, 4: Easy  
    delta_t INTEGER NOT NULL, -- Days elapsed since previous review  
    reviewed_at INTEGER NOT NULL,  
    FOREIGN KEY(card_id) REFERENCES cards(id) ON DELETE CASCADE  
);
```

```
CREATE INDEX IF NOT EXISTS idx_review_logs_card ON  
review_logs(card_id);
```

To prevent UI thread locks during heavy database operations—such as fetching large card decks or executing historical FSRs parameter optimizations—all database access occurs inside asynchronous database worker actors or dedicated background tasks.

Operational Best Practices, Thread Management, & Hardware Acceleration

Designing a stable, multi-threaded application combining native UI execution with machine learning demands clear execution boundaries. Allocating compute-heavy operations (e.g., ONNX model inference, Rayon optimization loops) directly onto the main webview thread or Tokio async runtime thread starves UI rendering and causes micro-stuttering.

To maintain total system stability, execution responsibilities are partitioned across four specialized thread domains:

- **Thread Domain 1 (Main OS & UI Event Loop):** Dedicated strictly to hosting the Tauri webview instance, handling native operating system window events, servicing system tray interactions, and routing primary user input events.
- **Thread Domain 2 (Async I/O Runtime - Tokio):** Manages asynchronous file system calls, non-blocking SQLite database transactions, system timer events, and raw buffer serialization over `tauri::ipc::Channel` instances.
- **Thread Domain 3 (Dedicated Neural Compute Pool):** Enforces single-threaded sequential execution of ONNX Runtime models via blocking OS threads. Isolating Wav2Vec2 ASR and Piper TTS sessions to dedicated worker threads prevents resource contention over shared GPU memory and execution providers.
- **Thread Domain 4 (Parallel CPU Compute Pool - Rayon):** Offloads CPU-intensive mathematical calculations, such as running offline FSRs parameter fitting across thousands of review logs or executing Cost ADR simulations, preventing background optimization tasks from blocking real-time I/O or rendering.

Packaging and Dynamic Provider Fallback Strategies

- **Minimal Dynamic Libraries:** When bundling ort in release builds, enable the `download-binaries` feature or utilize the `load-dynamic` strategy to load ONNX Runtime dynamic libraries (`.dll`, `.so`, `.dylib`) dynamically at runtime.
- **ONNX Minimal Build Capabilities:** For strictly constrained edge distributions, compile ONNX Runtime binaries using `minimal-build` to eliminate unnecessary operator kernels, RTTI overhead, and string names, significantly reducing total application binary size.
- **Execution Provider Dynamic Fallbacks:** Hardware environments vary across user installations. Applications must probe for hardware capabilities at startup, attempting initialization in the following explicit priority order: `CUDA` -> `CoreML` -> `DirectML` -> `OpenVINO` -> `Standard CPU`. If a specialized EP initialization fails, the runtime catches the initialization error gracefully and falls back to CPU execution without crashing.

Implementation Summary & Subsystem Handoff Checklist

To maintain architectural integrity when implementing this platform, developers must ensure all subsystem handoff criteria are fulfilled across the codebase:

- Verify that all required Tauri v2 IPC channels, commands, and plugin capabilities are explicitly whitelisted inside `src-tauri/[span_145](start_span)[span_145](end_span)capabilities/ manifests`.
- Ensure raw PCM audio feeds from the microphone and generated TTS wave streams use `tauri::ipc::Channel` rather than standard JSON invoke payloads.
- Configure Cargo dependencies for `ort` with `default-features = false`, selectively enabling required features (`download-binaries` or `load-dynamic`, required EPs, half-precision float support) to avoid bringing unused zoo models into production binaries.
- Implement `fsrs-rs` scheduler logic using standard 21-parameter structs. Ensure parameter optimizations (`compute_parameters`) run asynchronously off the main thread using Rayon thread pools.
- Embed `harper-core` within a native Rust worker thread to provide instantaneous, zero-network grammar linting on all transcribed text before storing or displaying results.
- Configure embedded SQLite connections with WAL mode enabled and declarative migrations managed at application startup.

Works cited

1. fbilhaut/orp: A Lightweight Framework for Building ONNX Runtime, <https://github.com/fbilhaut/orp> 2. darjusul/wav2vec2-ONNX-collection - Hugging Face, <https://huggingface.co/darjusul/wav2vec2-ONNX-collection> 3. open-spaced-repetition/fsrs-rs: FSRS for Rust, including ... - GitHub, <https://github.com/open-spaced-repetition/fsrs-rs> 4. harper-core - crates.io: Rust Package Registry, <https://crates.io/crates/harper-core/0.62.0> 5. Issues · Automattic/harper - GitHub, <https://github.com/Automattic/harper/issues> 6. Tauri 2.0 Stable Release, <https://v2.tauri.app/blog/tauri-20/> 7. ort - crates.io: Rust Package Registry, <https://crates.io/crates/ort> 8. Rust, ORT, ONNX: Real-Time YOLO on a Live Webcam- Part 1, <https://medium.com/@alfred.weirich/rust-ort-onnx-real-time-yolo-on-a-live-webcam-part-1-b6edfb50bf9b> 9. Inter-Process Communication - Tauri, <https://v2.tauri.app/concept/inter-process-communication/> 10. piper-rs — Rust audio library // Lib.rs, <https://lib.rs/crates/piper-rs> 11. GitHub - tauri-apps/tauri-plugin-sql: [READ ONLY] This repository is, <https://github.com/tauri-apps/tauri-plugin-sql> 12. silvermine/tauri-plugin-sqlite - GitHub, <https://github.com/silvermine/tauri-plugin-sqlite> 13. Tauri V2 Overview, <https://huakun.tech/Full-Stack/Framework/Tauri/tauri-v2> 14. Configuration - Tauri, <https://v2.tauri.app/reference/config/> 15. IPC in Tauri — Tauri Commands vs Custom IPC, What to Use When, <https://dev.to/hiyoyok/ipc-in-tauri-tauri-commands-vs-custom-ipc-what-to-use-when-2ab4> 16. Calling the Frontend from Rust | Tauri, <https://v2.tauri.app/develop/calling-frontend/> 17. [feat] Additionally support pushing array buffers with the event system., <https://github.com/tauri-apps/tauri/issues/13405> 18. tauri-plugin-broadcast - crates.io: Rust Package Registry, <https://crates.io/crates/tauri-plugin-broadcast> 19. ort - Rust - Docs.rs, <https://docs.rs/ort> 20. onnxruntime - Rust - Docs.rs, <https://docs.rs/onnxruntime> 21. ort - ONNX Runtime Rust bindings - Crates.io, <https://crates.io/crates/ort/1.14.0> 22. piper-tts-rs - Rust Package Registry - Crates.io, <https://crates.io/crates/piper-tts-rs> 23. R bindings for fsrs-rs spaced repetition algorithm · GitHub, <https://github.com/open-spaced-repetition/r-fsrs> 24. Releases · open-spaced-repetition/fsrs-rs - GitHub, <https://github.com/open-spaced-repetition/fsrs-rs/releases> 25. SQL - Tauri,

<https://v2.tauri.app/plugin/sql/> 26. Tauri 2.0 - Sqlite DB - React - DEV Community,
<https://dev.to/focuscookie/tauri-20-sqlite-db-react-2aem> 27. Building a todo app in Tauri with
SQLite and sqlx, <https://tauritutorials.com/blog/building-a-todo-app-in-tauri-with-sqlite-and-sqlx>